

Past-Future Temporal Bottleneck for Memory-Augmented Future-Aware Vision-Language-Action Control

核心思想

现有 Vision-Language-Action 模型通常根据当前观测、语言指令和机器人状态直接生成未来动作。这种方式在当前帧信息充分、任务状态清晰的场景中效果较好，但在真实机器人操作中，当前帧往往不足以支持稳定决策。许多关键状态依赖于时间上下文，例如物体是否刚刚发生接触、机器人手部是否产生滑动、遮挡前物体的位置、当前动作处于抓取前还是放置后阶段等。因此，机器人策略不仅需要理解“当前看到什么”，还需要建模“过去发生了什么”。

另一方面，机器人动作生成也不应只停留在对当前状态的反应式预测。一个有效的策略还需要隐式推理未来任务进展，例如接下来是否会接触物体、是否会完成抓取、是否会到达目标状态。直接预测未来图像代价较高，且会引入大量与控制无关的视觉细节。因此，我们希望在 latent space 中学习一种面向动作生成的 future-aware representation。

基于此，本文提出一个统一的 **Past-Future Temporal Bottleneck**。该框架在预训练 VLA policy 之上，引入一个统一的时序重采样模块，将过去观测和未来观测分别压缩成两类 compact temporal tokens：

PastMemory : history observations $\rightarrow$ Unified Temporal Resampler $\rightarrow H_{mem}$

FuturePosterior : future observations $\rightarrow$ Unified Temporal Resampler $\rightarrow Z_{post}$

其中， H_{mem} 是推理时可用的历史记忆，用于补充当前观测中缺失的时间上下文； Z_{post} 是训练时可用的未来后验隐变量，用于提供 privileged future supervision。

同时，模型学习一个 Future Latent Prior：

current observation + H_{mem} + language + robot state $\rightarrow Z_{prior}$

Z_{prior} 的目标是在看不到未来观测的情况下，仅根据当前观测、历史记忆、语言指令和机器人状态预测一个 future-aware latent。训练时，模型同时构造 prior branch 和 posterior branch：

Prior branch : current + H_{mem} + Z_{prior} $\rightarrow$ action prediction

Posterior branch : current + H_{mem} + Z_{post} $\rightarrow$ action prediction

posterior branch 能看到未来观测，因此可以形成更强的未来感知表示；prior branch 不能看到未来，因此通过 latent alignment 学习逼近 posterior branch 的 hidden representation。推理时，future observations、future posterior path 和 posterior branch 被全部移除，只保留：

history + current + language + state $\rightarrow$ H_{mem} $\rightarrow$ Z_{prior} $\rightarrow$ action prediction

因此，模型在部署时不需要未来观测，却能够利用训练阶段学到的 future-aware latent reasoning 能力。

主要贡献

本文的核心不是简单增加历史帧，也不是单独引入未来预测模块，而是提出一个统一的过去-未来时序瓶颈，将历史记忆和未来监督放入同一个 temporal latent interface 中。

1. Unified Past-Future Temporal Resampler

我们提出一个统一的时序重采样模块，同时处理过去观测和未来观测：

$H_{mem} = UTR(history\ observations, role = past)$

$Z_{post} = UTR(future\ observations, role = future)$

两者共享 temporal compression core，但使用不同的 role embedding、past/future queries、output projection。这样模型可以共享通用的时序压缩能力，同时保留过去记忆和未来后验的语义差异。

2. Memory-Conditioned Future Latent Prior

模型不仅使用历史记忆增强当前决策，还进一步基于历史记忆预测未来隐变量：

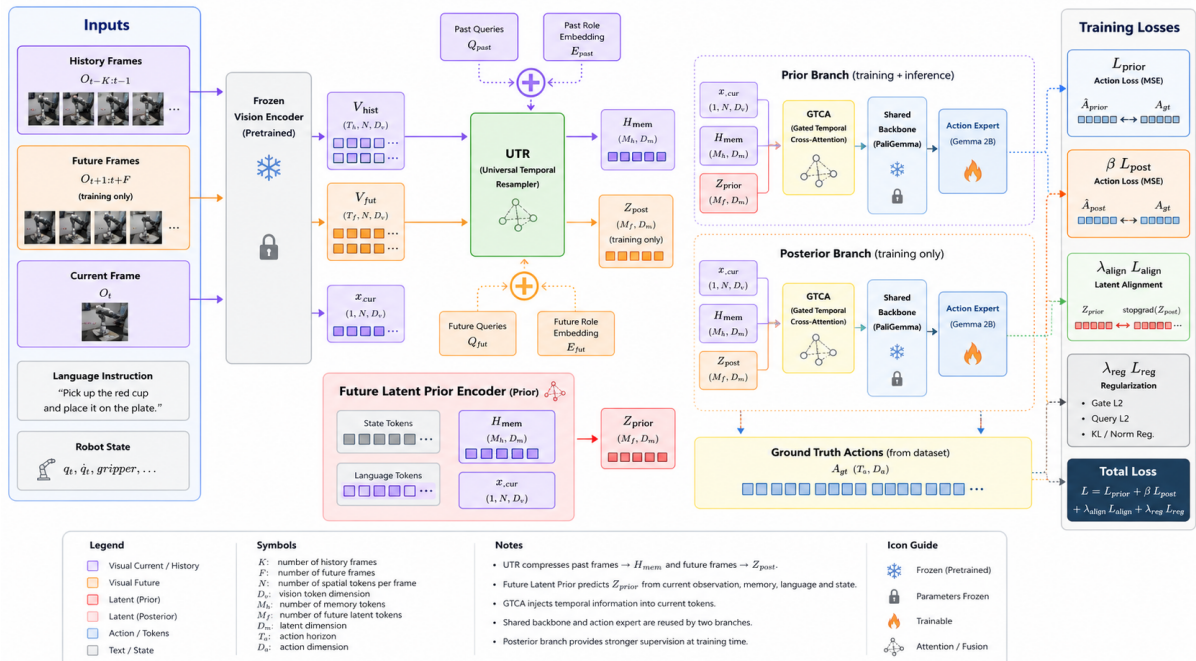
$$Z_{prior} = \text{Prior}(\text{current observation}, H_{mem}, \text{language}, \text{state})$$

这使得模型不只是“利用过去”，而是能够从过去和当前状态中推理未来任务进展。

3. Plug-in Temporal Injection via GTCA

基于 **Gated Temporal Cross-Attention (GTCA)** 的即插即用时序注入机制。我们设计了 Gated Temporal Cross-Attention 模块，将历史记忆 tokens 和未来隐变量 tokens 注入当前帧表示。GTCA 以当前动作预测特征 H_b^l 作为 Query，分别从历史记忆 H_{mem} 和 branch-specific future latent Z_b 中读取上下文信息，并通过两个独立 gate 控制历史信息和未来信息的注入强度。该设计无需从头训练或大幅修改 VLA 主干网络，可以在保持预训练 action backbone 稳定性的同时，引入历史记忆和未来推理能力，并为 prior/posterior 双分支提供统一的时序信息注入接口。

模型结构



如上图所示，本文框架在预训练 VLA policy 之上引入一个过去-未来统一时序瓶颈。训练阶段，模型输入包括历史帧、当前帧、语言指令、机器人状态以及未来帧。冻结视觉编码器首先提取历史、当前和未来观测的视觉 tokens。随后，Unified Temporal Resampler(UTR)，将历史视觉 tokens 压缩为过去记忆 tokens H_{mem} ，并将未来视觉 tokens 压缩为训练期后验隐变量 Z_{post} 。其中， H_{mem} 在训练和推理阶段均可使用，而 Z_{post} 只在训练阶段作为未来监督信号存在。

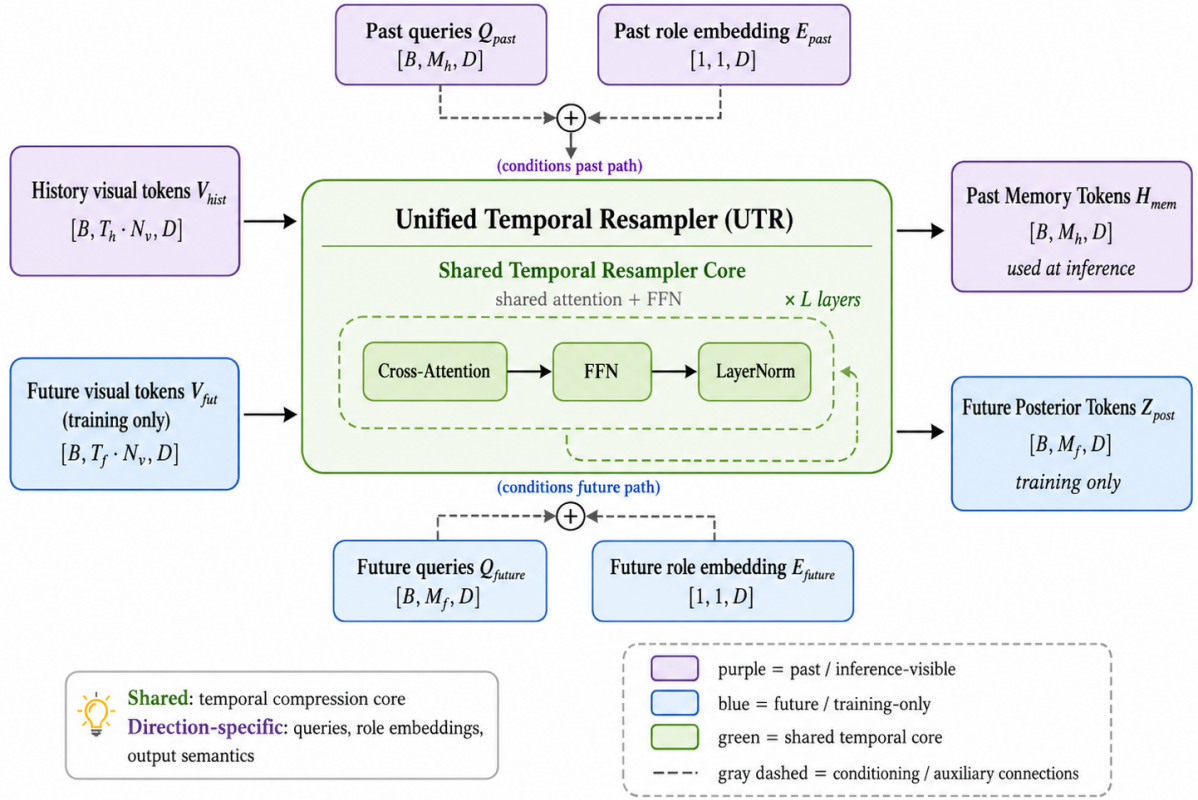
为了使模型在推理阶段也具备未来感知能力，我们进一步引入 Future Latent Prior Encoder，从当前视觉 tokens、历史记忆、语言 tokens 和机器人状态中预测未来先验隐变量 Z_{prior} 。随后，模型通过 Gated Temporal Cross-Attention(GTCA)，将历史记忆和未来隐变量注入当前帧表示。具体来说，GTCA 以当前动作预测特征 H_b^l 作为 Query，分别从历史记忆 H_{mem} 和 branch-specific future latent Z_b 中读取上下文信息，并通过两个独立 gate 控制历史信息和未来信息的注入强度。

训练时，prior branch 使用 x_{cur} 、 H_{mem} 和 Z_{prior} 进行动作预测，posterior branch 使用 x_{cur} 、 H_{mem} 和 Z_{post} 进行动作预测。两个分支共享动作生成主干，并通过动作损失和 latent alignment loss 共同优化。推理时，未来帧、 Z_{post} 和 posterior branch 被移除，模型仅依赖当前观测、历史记忆、语言、状态以及预测得到的未来隐变量生成动作。

Unified Past-Future Temporal Resampler

原始视觉序列通常包含大量冗余 token，如果直接将 past 和 future 的所有视觉 token 输入后续策略网络，会带来较高计算开销，也容易引入无关信息。因此，需要一个统一的时序重采样模块，将不同时间方向的视觉上下文压缩成固定数量的 memory token。本文提出 **Unified Temporal Resampler, UTR**，用于在统一框架下同时建模历史记忆和未来后验监督。与分别设计一个 history resampler 和一个 future resampler 不同，UTR 使用同一个 temporal resampler core 来处理过去和未来两类视觉 tokens。其目标是共享通用的时序压缩能力，同时通过方向特定的 queries 和 role embeddings 保留 past memory 与 future posterior 的语义差异。

给定历史视觉tokens和未来的tokens:



$$V_{hist} \in R^{B \times T_h \times N_v \times D_v}, V_{fut} \in R^{B \times T_f \times N_v \times D_v}.$$

首先将时间维度和空间 token 维度展平，并投影到统一 hidden dimension：

$$X_{hist} = Proj_v(flatten(V_{hist})) + E_{hist}^{time},$$

$$X_{fut} = Proj_v(flatten(V_{fut})) + E_{fut}^{time}.$$

然后，UTR 对 past 和 future 使用同一个共享函数：

$$Y_d^L = ResamplerCore_{\theta}(Y_d^0, X_d), d \in \{past, future\}.$$

其中， X_d 表示方向 d 对应的时序视觉 tokens， d 可以是历史或未来； Y_d^0 表示方向特定的可学习 latent queries，并加入对应的 role embedding。共享的 resampler core 通过多层 cross-attention 使 Y_d^0 读取 X_d 中的时序信息，得到压缩后的 latent tokens Y_d^L 。对于历史路径， Y_d^L 进一步投影为历史记忆 H_{mem} ；对于未来路径， Y_d^L 进一步投影为未来后验隐变量 Z_{post} 。其中 θ 表示共享的 resampler core 参数。也就是说，past path 和 future path 调用的是同一个 **ResamplerCore**，而不是两个独立模块。

对于历史路径：

$$Y_{past}^0 = Q_{past} + E_{past}$$

$$H_{mem} = Proj_{past}(ResamplerCore_{\theta}(Y_{past}^0, X_{hist}))$$

对于未来路径：

$$Y_{fut}^0 = Q_{fut} + E_{fut}$$

$$Z_{post} = Proj_{fut}(ResamplerCore_{\theta}(Y_{fut}^0, X_{fut}))$$

其中， Q_{past}, Q_{fut} 分别为带方向的可学习的query，用于提取历史或未来信息； E_{past}, E_{fut} 分别为方向特征embedding，用于编码方向； $Proj_{past}, Proj_{fut}$ 为区分方向的output projection；而 $ResamplerCore_{\theta}$ 为共享的时序压缩核心。

UTR共享模块的关系如下：

组成部分	Past分支	Future分支	是否共享	说明
输入图像序列	V_{hist}	V_{fut}	不共享	来自不同时间段，分别表示历史帧与未来帧
视觉投影层	$Proj_v(V_{past})$	$Proj_v(V_{fut})$	共享	使用同一投影层将视觉 token 映射到统一特征空间
投影后特征	X_{hist}	X_{fut}	不共享	参数共享，但由于输入不同，得到的特征表示不同
时间与方向编码	$TPE_{past} + E_{past}$	$TPE_{fut} + E_{fut}$	部分共享	时间编码生成方式共享，但具体时间位置和方向嵌入不同。
初始查询token	Y_{past}^0	Y_{fut}^0	不共享	不同方向使用方向相关的查询 token，以区分 past / future 语义
UTR core参数	$ResamplerCore_{\theta}$	$ResamplerCore_{\theta}$	共享	两个分支调用同一套 resampler 参数，包括 Attention、FFN 和 LayerNorm

组成部分	Past分支	Future分支	是否共享	说明
输出memory token	Y_{past}^L	Y_{fut}^L	不共享	分别得到历史压缩记忆和未来上下文记忆

ResamplerCore

在获得方向相关的视觉输入 token X_d 和初始查询 token Y_d^0 后，UTR 进一步通过共享的 ResamplerCore 对长时序视觉特征进行压缩，其中 $d \in \text{past, future}$ 表示时间方向。ResamplerCore 由 L 层重采样块堆叠而成，每一层主要包含Cross-Attention、FFN以及残差连接和LayerNorm。其核心作用是让少量token Y_d 从大量视觉token X_d 中选择并聚合关键时序信息，最终形成固定长度的memory token。

对于第 I 层Resampler block，首先将上一层的查询token Y_d^{l-1} 作为Query，将视觉输入token X_d 作为Key和Value，执行cross-attention：

$$Q_d^l = \text{LN}(Y_d^{l-1})W_Q,$$

$$K_d^l = \text{LN}(X_d)W_K,$$

$$V_d^l = \text{LN}(X_d)W_V.$$

随后计算注意力权重，并从视觉 token 中聚合信息：

$$\text{Attn}_d^l = \text{Softmax}\left(\frac{Q_d^l(K_d^l)^\top}{\sqrt{D_h}}\right)V_d^l,$$

其中， D_h 表示单个注意力头的特征维度。经过输出映射后，得到当前层的跨注意力更新结果：

$$\hat{Y}_d^l = Y_d^{l-1} + \text{Attn}_d^l W_O.$$

接着，Resampler block 通过 FFN 对查询 token 进行非线性变换，并结合残差连接得到当前层输出：

$$Y_d^l = \hat{Y}_d^l + \text{FFN} \left(\text{LN}(\hat{Y}_d^l) \right).$$

经过 L 层迭代后，最终得到方向相关的压缩 memory 表示：

$$Y_d^L = \text{ResamplerCore}_\theta(Y_d^0, X_d), \quad d \in \text{past, future}.$$

其中， θ 表示 ResamplerCore 的全部可学习参数，包括 cross-attention 中的 W_Q, W_K, W_V, W_O 、FFN 参数以及 LayerNorm 参数。需要注意的是，past 分支和 future 分支并不使用两套独立的 ResamplerCore，而是共享同一组参数 θ 。也就是说：

$$Y_{past}^L = \text{ResamplerCore}_\theta(Y_{past}^0, X_{past}),$$

$$Y_{fut}^L = \text{ResamplerCore}_\theta(Y_{fut}^0, X_{fut}).$$

这种设计使得 UTR 能够以统一的重采样机制处理不同时间方向的视觉上下文。虽然两个分支共享 ResamplerCore 参数，但由于它们的输入 token、方向编码和初始 query token 不同，最终得到的 Y_{past}^L 和 Y_{fut}^L 仍然保持不同的时间语义。换言之，ResamplerCore 共享的是“如何从长时序视觉 token 中提取关键信息”的处理规则，而不是共享具体的 past / future 表示。

从功能上看，ResamplerCore 可以被理解为一个由少量查询 token 驱动的时序信息压缩器。原始视觉输入 X_d 通常包含 $T_d \times N_v$ 个 token，其中 T_d 表示时间帧数， N_v 表示每帧视觉 token 数量。直接将这些 token 输入后续模块会带来较高的计算开销。相比之下，ResamplerCore 通过 cross-attention 将其压缩为固定数量的 memory token，不仅保留了关键时序上下文信息，还显著降低了后续模块的计算负担。

UTR 模块采用“参数共享、表示分离”的设计，使其能够在保证 past 与 future 语义独立的同时，复用统一的时序信息压缩机制。具体而言，该设计主要具有以下几个优势。

首先，UTR 能够有效降低时序视觉建模的计算开销。原始视觉输入通常包含大量时空 token，若直接将所有历史帧和未来帧 token 输入后续策略网络，会显著增加

attention 计算复杂度。设某一方向的视觉 token 数量为 $T_d N_v$ ，其中 T_d 表示时间帧数， N_v 表示每帧视觉 token 数量。ResamplerCore 通过 cross-attention 将其压缩为固定数量的 memory token：

$$Y_d^L \in \mathbb{R}^{B \times M \times D}, \quad M \ll T_d N_v.$$

因此，后续模块不再需要直接处理完整的长时序视觉序列，而只需关注压缩后的 memory token，从而降低计算成本并提升推理效率。

其次，参数共享设计提升了模型的参数效率。Past 分支和 future 分支并未分别引入两套独立的 resampler，而是共享同一组核心参数 θ ，这种方式使模型能够学习一种通用的时序重采样规则，即如何从长时间跨度的视觉 token 中提取关键上下文信息。相比为不同时间方向分别设计独立模块，共享参数不仅减少了模型规模，也降低了过拟合风险，尤其适用于训练数据规模有限的场景。

第三，UTR 在共享参数的同时保留了 past 与 future 的语义差异。虽然两个分支使用相同的 ResamplerCore，但它们的输入视觉序列、时间位置编码、方向嵌入以及初始查询 token 均保持独立。因此，UTR 并不是将历史信息和未来信息简单混合，而是分别对两个时间方向执行重采样，得到具有不同时间语义的 memory 表示：

$$Y_{past}^L \neq Y_{fut}^L.$$

其中， Y_{past}^L 主要用于表示历史上下文信息，帮助模型理解过去的状态变化； Y_{fut}^L 则用于表示未来相关上下文或目标轨迹信息，为后续动作预测提供前瞻性参考。这样的设计能够避免 past 与 future 信息在表示层面发生混淆。

第四，统一的重采样结构有利于提升模型的泛化能力。由于 past 和 future 共享同一套特征压缩机制，模型在训练过程中能够从两个时间方向共同学习稳定的时序聚合模式。也就是说，UTR 学到的不是某一特定方向的局部处理规则，而是一种更通用的视觉时序信息抽取方式。这种统一建模方式有助于模型在不同任务、不同时间跨度或不同轨迹模式下保持更好的适应性。

综上，UTR 的设计在计算效率、参数效率和语义解耦之间取得了平衡。它通过共享 ResamplerCore 参数降低模型复杂度，通过方向相关输入和查询 token 保持

past / future 表示独立，并通过固定数量的 memory token 减轻后续策略网络的计算负担。因此，UTR 可以作为一种高效且通用的时序视觉压缩模块，为后续的动作预测和决策生成提供更加紧凑、稳定的上下文表示。

Future Latent Prior Encoder

在训练阶段，模型可以访问未来帧 $O_{t+1:t+F}$ ，并通过 UTR 将其压缩为 posterior latent 表示 Z_{post} 。该表示包含了未来视觉变化的信息，因此能够为动作预测提供较强的监督信号。然而，在实际推理阶段，未来帧不可获得，模型只能基于历史观测、当前观测、语言指令和机器人状态进行决策。因此，不能直接依赖 Z_{post} 进行动作预测。

为了解决训练阶段和推理阶段之间的信息不一致问题，本文引入 **Future Latent Prior Encoder**。该模块不直接使用未来帧，而是根据当前可用信息预测一个未来相关的 latent 表示 Z_{prior} 。在训练阶段， Z_{prior} 会受到 Z_{post} 的对齐监督；在推理阶段，模型仅使用 Z_{prior} 完成动作预测。

Future Latent Prior Encoder 的输入由四部分组成：

$$x_{cur}, H_{mem}, E_{lang}, E_{state}$$

其中， x_{cur} 表示当前帧视觉 token， H_{mem} 表示由 UTR 得到的历史记忆 token， E_{lang} 表示语言指令 token， E_{state} 表示机器人状态 token。该模块的输出为：

$$Z_{prior} \in \mathbb{R}^{B \times M_z \times D_m}$$

其中， M_z 表示 future latent token 的数量， D_m 表示 latent token 的特征维度。

整体可以表示为：

$$Z_{prior} = \text{PriorEncoder}_{\phi} \left(x_{cur}, H_{mem}, E_{lang}, E_{state} \right),$$

其中， ϕ 表示 Future Latent Prior Encoder 的可学习参数。

为了从当前状态中预测未来 latent，首先将不同模态的信息投影到统一特征空间。对于当前帧视觉 token、语言 token 和状态 token，分别进行线性投影：

$$\bar{x}_{cur} = P_x(x_{cur}), \quad \bar{E}_{lang} = P_l(E_{lang}), \quad \bar{E}_{state} = P_s(E_{state}), \quad \bar{H}_{mem} = P_h(H_{mem}).$$

其中， P_x 、 P_l 、 P_s 和 P_h 是模态投影层，用于将不同来源的 token 对齐到相同维度 D_m 。随后，将这些条件信息与历史记忆 token 拼接，得到 prior encoder 的条件上下文：

$$C_{prior} = \text{Concat}(\bar{x}_{cur}, \bar{H}_{mem}, \bar{E}_{lang}, \bar{E}_{state}).$$

这里， C_{prior} 同时包含当前视觉状态、历史时序上下文、任务语义和机器人本体状态，是预测未来 latent 的主要依据。

Future Latent Prior Encoder 使用一组可学习的 future latent query 作为初始输入：

$$Z_{prior}^0 = Q_{prior},$$

其中：

$$Q_{prior} \in \mathbb{R}^{1 \times M_f \times D_m}.$$

在训练和推理时， Q_{prior} 会沿 batch 维度复制，并通过 cross-attention 从条件上下文 C_{prior} 中读取信息。对于第 l 层 prior encoder block，其计算过程为：

$$\tilde{Z}_{prior}^l = Z_{prior}^{l-1} + \text{CrossAttn}\left(\text{LN}(Z_{prior}^{l-1}), \text{LN}(C_{prior})\right),$$

$$Z_{prior}^l = \tilde{Z}_{prior}^l + \text{FFN}\left(\text{LN}(\tilde{Z}_{prior}^l)\right).$$

经过 L_p 层堆叠后，得到最终的 future prior latent：

$$Z_{prior} = Z_{prior}^{L_p}.$$

该过程可以理解为：模型以一组 future latent query 为载体，从当前帧、历史记忆、语言指令和机器人状态中抽取与未来动作演化相关的信息，从而生成一个不依赖未来帧的 latent 表示。

Future Latent Prior Encoder 预测的是 Z_{prior} ，而 UTR 通过未来帧得到的是 Z_{post} 。二者的来源不同：

$$Z_{post} = UTR(V_{fut}),$$

$$Z_{prior} = PriorEncoder_{\phi}(x_{cur}, H_{mem}, E_{lang}, E_{state}).$$

其中， Z_{post} 直接由未来帧得到，因此只在训练阶段可用；而 Z_{prior} 由当前可观测信息预测得到，因此训练和推理阶段均可使用。

为了让 Z_{prior} 学习到未来相关的信息，训练时引入 latent alignment loss，使其向 Z_{post} 靠近：

$$\mathcal{L}_{align} = \left\| Z_{prior} - sg(Z_{post}) \right\|_2^2$$

其中， $sg(\cdot)$ 表示 stop-gradient 操作。该设计使 Z_{post} 作为训练阶段的 latent teacher，为 Z_{prior} 提供未来信息监督；同时避免 posterior 分支被 prior 分支反向影响，从而保持 Z_{post} 的稳定性。

Posterior Latent Encoder 的核心作用是在训练阶段利用未来帧构造具有前瞻信息的 latent 表示。具体而言，该模块以未来视觉序列 V_{fut} 作为输入，通过 UTR 对未来时序视觉 token 进行压缩，得到 posterior latent。由于 Z_{post} 直接由未来帧生成，它能够显式编码未来状态变化、目标接近过程以及动作执行后的视觉结果。因此，相比仅依赖当前帧和历史记忆，posterior latent 能够提供更强的未来语义监督。

需要注意的是， Z_{post} 只在训练阶段可用。在推理阶段，模型无法访问未来帧，因此不能直接使用 posterior latent 进行动作预测。为了解决这一问题，本文进一步设

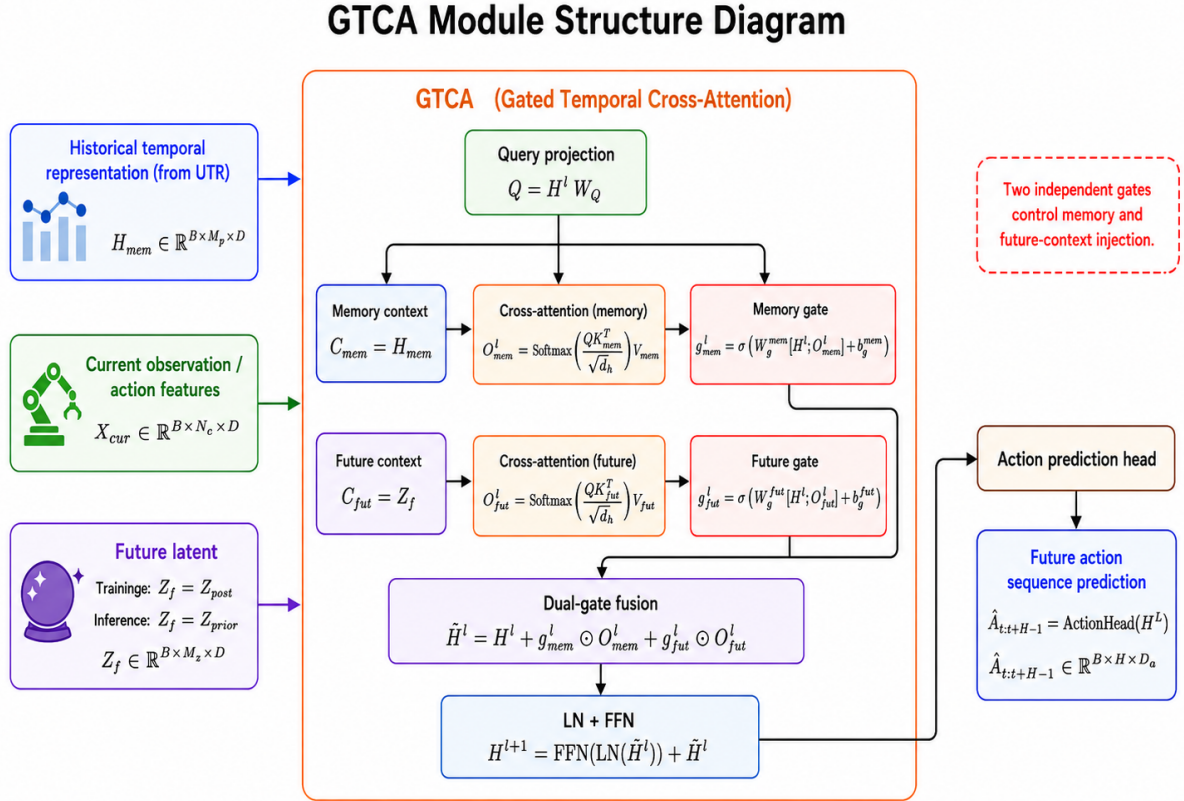
计 Future Latent Prior Encoder，从当前观测、历史记忆、语言指令和机器人状态中预测 Z_{prior} ，并通过 latent alignment loss 使其向 Z_{post} 对齐。这样， Z_{post} 可以作为训练阶段的 latent teacher，引导 Z_{prior} 学习未来相关信息。在 latent alignment 中，stop-gradient 作用于 Z_{post} ，因此该损失只更新 prior encoder，而不会通过该项反向更新 future posterior path。

总体而言，Posterior Latent Encoder 并不直接作为推理模块使用，而是作为训练阶段的未来信息编码器，为 prior 分支提供监督信号。它使模型能够在训练时利用未来帧学习更具前瞻性的表示，同时保证推理阶段仍然只依赖可观测信息完成动作预测。然而，仅获得 H_{mem} 、 Z_{post} 或 Z_{prior} 还不足以完成动作生成。模型还需要将历史记忆和未来 latent 有效注入当前帧表示中，使当前视觉 token 同时具备历史上下文和未来预测信息。为此，本文进一步引入 Gated Temporal Cross-Attention (GTCA) 模块。GTCA 在 prior branch 和 posterior branch 中分别执行时序上下文融合，通过两个独立 gate 控制历史记忆和未来 latent 对当前动作预测特征的影响强度，从而在增强时序建模能力的同时，避免过度扰动预训练动作生成主干。

Gated Temporal Cross-Attention

GTCA (Gated Temporal Cross-Attention) 模块的作用是在动作预测之前，对当前观测特征、历史时序记忆以及未来 latent 表示进行进一步融合。前面的 UTR 模块已经将历史帧和未来帧分别压缩为统一的时序 token 表示，而 Future Latent Prior Encoder 则在没有未来帧的情况下预测未来相关 latent。GTCA 模块进一步将这些信息注入当前决策表征中，使动作预测网络能够同时关注当前状态、历史动态变化以及未来趋势信息。

如图所示，GTCA用于在动作预测阶段融合历史时序信息和未来趋势信息。前面的 UTR 模块已经将历史视觉 token 压缩为历史时序表示 H_{mem} ，Future Latent Prior Encoder 在训练和推理阶段均根据当前可观测信息预测 Z_{prior} 。训练阶段， Z_{prior} 通过 latent alignment 向 Z_{post} 对齐；推理阶段，仅保留 Z_{prior} 用于动作预测。



对于 prior branch 和 posterior branch，定义 branch-specific future latent：

$$Z_b = \begin{cases} Z_{prior}, & \text{prior branch,} \\ Z_{post}, & \text{posterior branch.} \end{cases}$$

其中，prior branch 在训练和推理阶段均可使用，而 posterior branch 仅在训练阶段用于提供未来监督信号。

$$X_{cur} \in \mathbb{R}^{B \times N_c \times D},$$

UTR 输出的历史记忆为：

$$H_{mem} \in \mathbb{R}^{B \times M_p \times D},$$

未来latent表示为：

$$Z_b \in \mathbb{R}^{B \times M_z \times D}.$$

GTCA 不再简单地将历史记忆和未来 latent 拼接为单一上下文，而是将二者作为两类独立的时序上下文来源。定义上下文类型索引：

$$r \in \{mem, fut\},$$

并令：

$$C_r = \begin{cases} H_{mem}, & r = mem, \\ Z_b, & r = fut. \end{cases}$$

其中， C_{mem} 表示历史时序记忆，用于提供过去状态变化、接触历史和遮挡前信息； C_{fut} 表示未来 latent，用于提供对后续状态演化和动作趋势的预测信息。

对于第 1 层 GTCA，设当前动作预测特征为：

$$H_b^l \in \mathbb{R}^{B \times N_a \times D},$$

其中， N_a 表示动作 token 或当前决策 token 的数量， $b \in \{prior, post\}$ 表示当前分支。GTCA 使用当前动作预测特征作为 Query，分别从历史记忆和未来 latent 中读取上下文信息：

$$Q_b^l = H_b^l W_Q,$$

$$K_r^l = C_r W_K^r,$$

$$V_r^l = C_r W_V^r.$$

随后，对每一类时序上下文执行 cross-attention：

$$O_r^l = \text{Softmax} \left(\frac{Q_b^l (K_r^l)^\top}{\sqrt{d_h}} \right) V_r^l, \quad r \in \{mem, fut\}.$$

其中, O_{mem}^l 表示从历史记忆中读取到的上下文特征, O_{fut}^l 表示从未来 latent 中读取到的上下文特征。相比直接拼接时序信息, 这种双分支 cross-attention 使动作 token 能够分别选择与当前决策最相关的历史片段和未来趋势语义。

为了避免额外时序上下文在训练初期对动作预测造成过强干扰, GTCA 为两类上下文分别引入独立的门控机制:

$$g_r^l = \sigma \left(W_g^r [H_b^l; O_r^l] + b_g^r \right), \quad r \in \{mem, fut\}.$$

其中, $\sigma(\cdot)$ 表示 Sigmoid 函数, $[\cdot; \cdot]$ 表示特征拼接, g_{mem}^l 控制历史记忆的注入强度, g_{fut}^l 控制未来 latent 的注入强度。随后, 通过双门控残差更新当前动作预测特征:

$$\tilde{H}_b^l = H_b^l + \sum_{r \in \{mem, fut\}} g_r^l \odot O_r^l.$$

其中, $\odot$ 表示逐元素乘法。最后, 经过归一化和前馈网络得到下一层输出:

$$H_b^{l+1} = \text{FFN} \left(\text{LN}(\tilde{H}_b^l) \right) + \tilde{H}_b^l.$$

经过多层 GTCA 更新后, 增强后的动作预测特征被送入动作预测头, 生成未来动作序列:

$$\hat{A}_{t:t+H-1}^b = \text{ActionHead}(H_b^L),$$

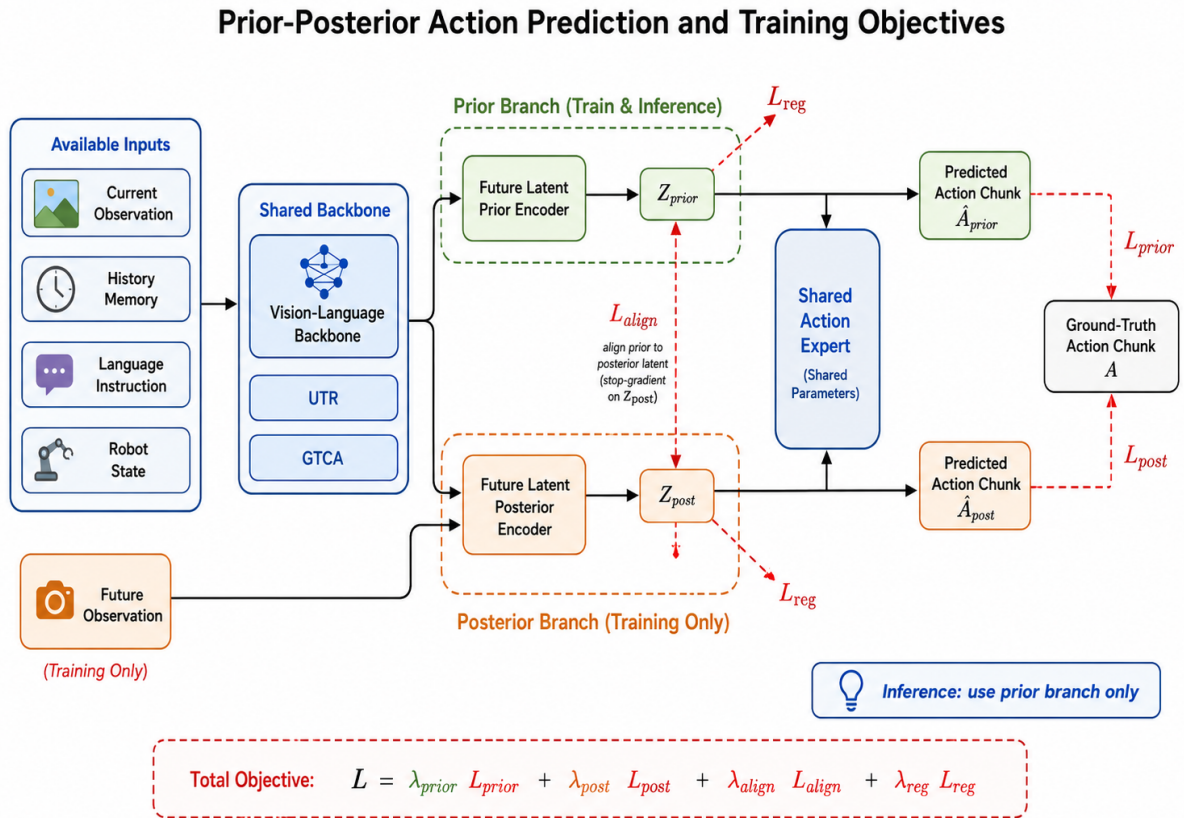
$$\hat{A}_{t:t+H-1}^b \in \mathbb{R}^{B \times H \times d_a}.$$

其中, H 表示动作预测 horizon, d_a 表示单步动作维度。当 $b = prior$ 时, 模型使用 Z_{prior} 进行动作预测; 当 $b = post$ 时, 模型使用 Z_{post} 进行训练期辅助预测。推理阶段仅保留 prior branch, 即:

$$\hat{A}_{t:t+H-1} = \hat{A}_{t:t+H-1}^{prior}.$$

GTCA 的核心作用可以概括为三点：第一，它将历史动态信息和未来趋势语义作为两类独立上下文进行建模，使动作预测不再只依赖当前帧；第二，它通过 cross-attention 让当前动作 token 分别查询历史记忆和未来 latent，提高上下文选择能力；第三，它通过两个独立 gate 自适应控制历史信息和未来信息的注入强度，从而提升训练稳定性，并减少对原始动作预测主干的扰动。

Prior-Posterior Action Prediction and Training Objectives



在完成 UTR 历史信息聚合、Future Latent Prior/Posterior 表征建模以及 GTCA 跨时序上下文融合之后，模型进一步采用 prior-posterior 双分支动作预测机制进行训练。该机制的核心思想是：在训练阶段同时构建 prior 分支和 posterior 分支，其中 posterior 分支能够访问未来观测，因此可以提供更强的未来语义监督；而 prior 分支仅依赖当前观测、历史记忆、语言指令和机器人状态，这些信息在推理阶段均

可获得。因此，posterior 分支主要作为训练阶段的辅助教师分支，而 prior 分支则作为最终推理时实际使用的动作预测分支。

设 Z_{prior} 表示 Future Latent Prior Encoder输出的先验潜变量，而 Z_{post} 表示由 UTR 的 future path 编码未来帧得到的后验潜变量。为便于表述，本文也将该 future path 称为 posterior latent encoder，但其与历史路径共享同一个 ResamplerCore。经过 GTCA 融合后，prior 分支和 posterior 分支分别得到对应的上下文表征：

$$H_{prior} = B_{\theta} \left(X_{cur}, H_{mem}, q, s_t, Z_{prior} \right),$$

$$H_{post} = B_{\theta} \left(X_{cur}, H_{mem}, q, s_t, Z_{post} \right).$$

其中， q 表示语言指令， s_t 表示当前时刻的机器人状态， B_{θ} 表示共享的主干网络。该主干网络包含视觉-语言特征提取、历史记忆聚合以及 GTCA 上下文融合等模块。

随后，两个分支共享同一个 *action expert* E_{ϕ} 进行动作预测：

$$\hat{A}_{t:t+H-1}^{prior} = \mathcal{E}_{\phi} \left(H_{prior} \right),$$

$$\hat{A}_{t:t+H-1}^{post} = \mathcal{E}_{\phi} \left(H_{post} \right).$$

其中， $(\hat{A}^{prior}, \hat{A}^{post} \in \mathbb{R}^{B \times H \times d_a})$ 分别表示 prior 分支和 posterior 分支预测得到的动作序列， H 表示动作预测 horizon， d_a 表示单步动作维度。

这种设计的关键在于，prior 分支和 posterior 分支并不使用两套独立的动作预测网络，而是共享相同的 backbone 参数 θ 和 action expert 参数 ϕ 。因此，两个分支的动作预测被约束在同一个特征空间和动作预测空间中。posterior 分支由于能够访问未来观测，可以学习到更加明确的未来状态变化和动作趋势；而 prior 分支则通过与 posterior 分支进行 latent 对齐，在不访问未来帧的情况下学习预测未来相关信息。这样，模型既能在训练阶段利用未来信息增强监督，又能在推理阶段仅依赖可用输入完成未来感知的动作预测。

训练阶段的真实动作序列记为：

$$A_{t:t+H-1} = [a_t, a_{t+1}, \dots, a_{t+H-1}].$$

prior 分支通过真实动作序列进行监督，其动作预测损失定义为：

$$\mathcal{L}_{prior} = \frac{1}{BHd_a} \sum_{b=1}^B \sum_{i=0}^{H-1} \left\| \hat{a}_{t+i}^{b,prior} - a_{t+i} \right\|_2^2.$$

类似地，posterior 分支同样使用真实动作序列进行监督，其动作预测损失定义为：

$$\mathcal{L}_{post} = \frac{1}{BHd_a} \sum_{b=1}^B \sum_{i=0}^{H-1} \left\| \hat{a}_{t+i}^{b,post} - a_{t+i} \right\|_2^2.$$

其中， $\mathcal{L}_{prior}$ 直接优化推理阶段所使用的 prior 分支，使其能够根据当前观测和历史记忆预测未来动作； $\mathcal{L}_{post}$ 则优化训练阶段的 posterior 分支，使其充分利用未来观测学习更强的未来感知动作表征。

然而，由于 posterior 分支在推理阶段不可用，仅优化 posterior 分支并不能直接提升最终部署性能。因此，为了将 posterior 分支中的未来信息迁移到 prior 分支中，模型进一步引入 latent alignment loss，用于约束 Z_{prior} 接近 Z_{post} ：

$$\mathcal{L}_{align} = \frac{1}{BM_z D_z} \left\| P_{prior}(Z_{prior}) - sg \left(P_{post}(Z_{post}) \right) \right\|_2^2.$$

其中， $sg(\cdot)$ 表示 stop-gradient 操作， $sg(\cdot)$ 作用于投影后的 posterior latent $P_{post}(Z_{post})$ ，因此该 alignment loss 只更新 prior encoder 及 prior-side projection，而不会通过该损失反向更新 future posterior path。该操作使 Z_{post} 作为稳定的训练目标，引导 Z_{prior} 学习未来相关的潜在表示，同时避免 prior 分支和 posterior 分支之间发生不稳定的相互牵引。通过该对齐约束，posterior 分支可以被视为训练阶段的 teacher，而 prior 分支则学习在缺少未来帧的条件下近似 posterior latent 中包含的未来信息。

为了进一步约束 latent 表征的尺度，避免潜变量过度放大或出现不稳定分布，引入 latent regularization loss：

$$\mathcal{L}_{reg} = \frac{1}{BM_z D_z} (\|P_{prior}(Z_{prior})\|_2^2 + \|P_{post}(Z_{post})\|_2^2).$$

最终，模型的整体训练目标定义为：

$$\mathcal{L} = \lambda_{prior}\mathcal{L}_{prior} + \lambda_{post}\mathcal{L}_{post} + \lambda_{align}\mathcal{L}_{align} + \lambda_{reg}\mathcal{L}_{reg}.$$

其中， λ_{prior} 、 λ_{post} 、 λ_{align} 和 λ_{reg} 分别表示 prior 动作预测损失、posterior 动作预测损失、latent 对齐损失和 latent 正则项的权重系数。

通过上述训练目标，posterior 分支能够充分利用未来观测学习更强的动作预测表示，而 prior 分支则在动作监督和 latent 对齐监督的共同作用下，逐渐学习到与未来状态变化相关的先验潜变量表示。在推理阶段，未来观测不可用，因此模型仅保留 prior 分支进行动作预测：

$$\hat{A} = \hat{A}_{prior}.$$

因此，该 prior-posterior 双分支训练机制在不增加推理阶段额外输入的前提下，将训练阶段可获得的未来信息有效迁移到 prior 分支中，从而提升模型在实际部署时的动作预测能力和时序决策稳定性。